TP 4 - Compression d'images par transformée mathématique (*JPEG*)

Christophe TILMANT

1 Introduction - Compression par transformée

Dans le domaine de la compression de données, il existe deux approches possibles :

— *la compression sans pertes* ou *codage* qui réduit la taille des données en diminuant la redondance du codage de celles-ci. Lors de la phase de décompression, ou décodage, on se retrouve exactement avec les données de départ ;

— *la compression avec pertes* ou *compression* qui supprime des informations dans le signal, jugées non pertinentes pour l'utilisation a posteriori de ces données. Généralement, ces approches sont utilisées pour compresser des données de son, d'image, de vidéo, ... Lors de la décompression, on obtient un signal différent du signal original, mais avec une différence qu'on espère imperceptible pour un cas d'utilisation donné (écoute de la musique, visualisation de l'image, etc.).

Les méthodes de compression avec pertes se basent souvent sur des techniques de *compression par transformée*. Le principe consiste à appliquer une transformée au signal original, puis à quantifier les coefficients de la représentation obtenue dans l'espace transformé.

Plusieurs arguments justifient cette approche :

— La transformée, qui équivaut mathématiquement à un changement de base d'encodage du signal, permet souvent d'obtenir une représentation alternative du signal qui met en évidence son contenu important et de ce fait, les informations pertinentes à enregistrer ou à transmettre ;

— La transformée permet de concentrer l'énergie du signal uniquement sur quelques coefficients, en obtenant une représentation *parcimonieuse*, ce qui implique moins de données à enregistrer, en supprimant les coefficients de faibles valeurs dans la représentation transformée ;

— La transformée réduit la corrélation des données, et diminue donc l'entropie de la source. La théorie de l'information de Shannon [1] relie l'entropie de la source à la capacité de codage, donc de compression.

La Fig. 1 présente le concept général de la compression par transformée. En résumé, l'étape de quantification des coefficients de la transformée permet de réduire leur nombre car ceux trop proches de zéro ne seront pas conservés.

1. Shannon, C. E. (1948). A Mathematical Theory of Communication. *The Bell System Technical Journal*, 27, 379–423.

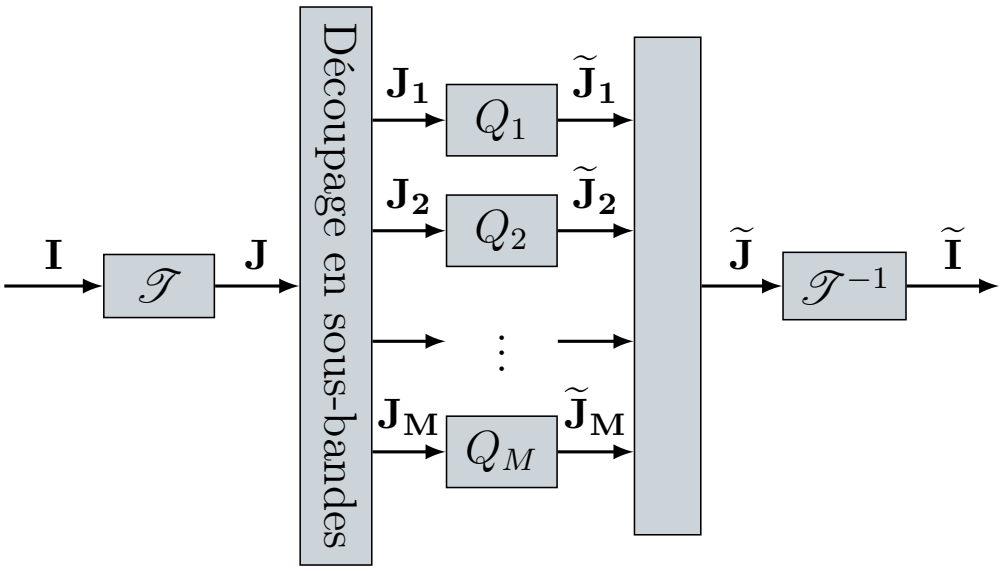

FIGURE 1 – Principe de la compression par transformée. Suite à une transformée $\mathscr{T}$, des sous-bandes sont quantifiées pour rendre parcimonieuse l'information et ainsi réduire la quantité de données à enregistrer $\left\{\widetilde{\mathbf{J}}_\mathbf{i}\right\}_{i=1..M}$.

2 Algorithme *JPEG*

La norme *JPEG*[2] définit le format et l'algorithme de décodage pour les images compressées avec pertes. *JPEG* est l'acronyme de *Joint Photographic Experts Group*, un rassemblement d'experts travaillant sur ces sujets. La norme officielle et définitive de la compression *JPEG* pour les images a été adoptée en 1992. Le processus de compression et de décompression *JPEG* comporte six étapes principales représentées sur la Fig. 2 :

1. **Transformation de couleurs** : *JPEG* travaille sur l'espace colorimétrique de luminance/chrominance $YCbCr$ (voir TP - Espaces Colorimétriques) ;
2. **Sous-échantillonnage de la chrominance** : pour exploiter la faible sensibilité de l'œil humain à la chrominance, on procède à un sous-échantillonnage des signaux de chrominance. Le principe de l'opération (décimation) est de réduire la taille des blocs de chrominance sans que celle des blocs de luminance (blocs Y) soit modifiée (différentes possibilités sont offertes : `4:4:4`, `4:2:0`, `4:2:2`, ...) ;
3. **Découpage en blocs** : le format *JPEG* découpe l'image en blocs carrés 8×8 (64 pixels chacun) ;
4. ***DCT*** : la transformée *DCT* (*Discrete Cosine Transform* en français, c'est-à-dire la transformée en cosinus discrète), est une transformation numérique appliquée à chaque bloc. C'est une variante réelle de la transformée de Fourier (TP - Filtrage Fréquentiel). Chaque bloc est ainsi converti en une carte de fréquences et d'amplitudes plutôt qu'en intensités lumineuses et coefficients de couleur. La valeur d'une fréquence dans un bloc reflète l'importance et la rapidité d'un changement dans une base de cosinus, tandis que la valeur d'amplitude correspond à l'écart associé à chaque changement de couleur. L'application de la *DCT* est une opération mathématique sans perte d'informations, donc inversible ;
5. **Quantification** : la quantification est l'étape de l'algorithme de compression *JPEG* au cours de laquelle se produit la majeure partie de la perte d'information (donc de la qualité

2. ISO/CEI 10918-1 | UIT-T Recommendation T.81

visuelle), mais c'est aussi celle qui permet de compresser le signal. La quantification consiste à diviser la matrice issue de la DCT à l'aide d'une autre matrice, appelée *matrice de quantification*, qui contient 8×8 coefficients spécifiquement choisis par la norme. Le but est ici d'atténuer les très hautes fréquences dans le bloc, c'est-à-dire celles auxquelles l'œil humain est le moins sensible. Ces fréquences ont initialement des amplitudes faibles, qui sont encore plus atténuées par la quantification (certains coefficients étant même souvent ramenés à 0) ;

6. **Codage RLE et Huffman** : le codage des coefficients DC (valeur de la DCT en zéro = valeur moyenne) et AC (les autres coefficients) sont différents. Les coefficients AC sont parcourus et codés en zigzag, puis compressés par un algorithme RLE (*Run Length Encoding*) et enfin par un codage entropique de type Huffman ou codage arithmétique. Les deux premières étapes permettent de diminuer l'entropie de la source et ainsi rendre plus performant le codage entropique.

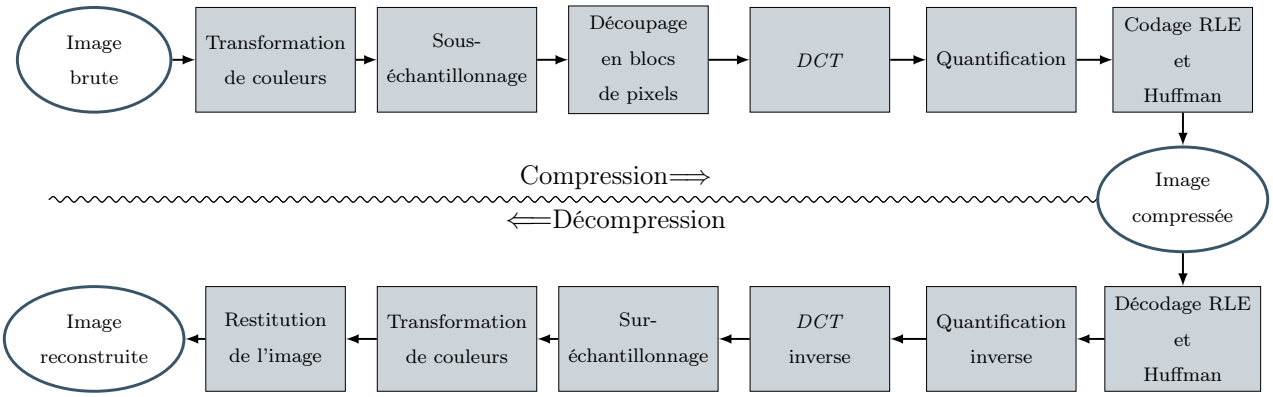

FIGURE 2 – Principe général de la compression et décompression *JPEG*.

3 Transformée en cosinus discrète et quantification

La transformée en cosinus discrète, notée J, d'une image numérique I de taille $N \times N$ pixels est définie pour tout $(u, v) \in [\![0, N-1]\!]^2$ par :

$$J[u,v] = \frac{2}{N} C[u]\, C[v] \sum_{i=0}^{N-1} \sum_{j=0}^{N-1} I[i,j] \cos\left[\frac{(2i+1)u\pi}{2N}\right] \cos\left[\frac{(2j+1)v\pi}{2N}\right]$$

$$\text{où } C[u] = \begin{cases} \frac{1}{\sqrt{2}} & \text{pour } u = 0 \\ 1 & \text{pour } u > 0 \end{cases}$$

Exercice 1 :

Transformée en cosinus discrète d'une image carrée : Concevez une fonction permettant de calculer la DCT d'une image carrée de taille $N \times N$. Affichez la transformée obtenue et interprétez les différentes valeurs.
Testez votre programme sur l'image `kingfischer.bmp`.

La transformée inverse est définie pour tout $(i,j) \in [\![0, N-1]\!]^2$ par :

$$I[i,j] = \frac{2}{N} \sum_{u=0}^{N-1} \sum_{v=0}^{N-1} C[u]\, C[v]\; J[u,v] \cos\left[\frac{(2i+1)u\pi}{2N}\right] \cos\left[\frac{(2j+1)v\pi}{2N}\right]$$

Exercice 2 :

Transformée en cosinus discrète inverse d'une image carrée : Concevez une fonction qui calcule la *DCT inverse* d'une image carrée de dimensions $N \times N$. Pour valider vos deux fonctions (*DCT directe* et *DCT inverse*), appliquez-les successivement sur la même image. Le résultat final doit permettre de retrouver l'image d'origine.
Testez votre programme en utilisant l'image `kingfischer.bmp`.

L'étape de quantification, s'écrit :

$$\forall (u,v) \in [\![0, N-1]\!]^2,\ \widetilde{J}[u,v] = \text{round}\left(\frac{J[u,v]}{\gamma \times Q[u,v]}\right).$$

où la matrice de quantification (issue de la norme *JPEG* et où $N = 8$) est donnée par :

$$\mathbf{Q} = \begin{bmatrix} 16 & 11 & 10 & 16 & 24 & 40 & 51 & 61 \\ 12 & 12 & 14 & 19 & 26 & 58 & 60 & 55 \\ 14 & 13 & 16 & 24 & 40 & 57 & 69 & 56 \\ 14 & 17 & 22 & 29 & 51 & 87 & 80 & 62 \\ 18 & 22 & 37 & 56 & 68 & 109 & 103 & 77 \\ 24 & 35 & 55 & 64 & 81 & 104 & 113 & 92 \\ 49 & 64 & 78 & 87 & 103 & 121 & 120 & 101 \\ 72 & 92 & 95 & 98 & 112 & 100 & 103 & 99 \end{bmatrix}.$$

Le paramètre γ permet de régler le taux de compression ou la qualité de l'image reconstruite. Plus γ est grand et plus l'image subira de pertes, amenant donc à un meilleur taux de compression. L'implémentation de l'étape de quantification dans l'espace transformé est donnée par le code 2. Afin d'optimiser le temps de calcul de la *DCT*, les valeurs des cosinus, qui restent constantes, peuvent être précalculées. Cette optimisation permet de réduire significativement la complexité des calculs lors de l'exécution. Cette étape est détaillée dans le code 1.

```
CImg<> genCosValues()
{
 CImg<> cosinusvalues(N,N);
 cimg_forXY(cosinusvalues,i,x)
  cosinusvalues(x,i) = std::cos(((2*x+1)*i*cimg::PI)/(2*N));
 return cosinusvalues;
}
```

Code source 1: Tabulation des valeurs du cosinus.

```cpp
/*
  JPEGEncoder: Calcul de l'image transformée par DCT + Quantification

  image     : Image d'entrée
  quality   : Facteur de qualité ( faible = meilleure qualité )
  cosvalues : Précalcul des valeurs du cosinus pour la DCT
  return    : Image des DCT quantifiées
*/
CImg<> JPEGEncoder(CImg<>& image, float quality, CImg<>& cosvalues)
{
  unsigned int N = 8; // Résolution d'un bloc

  // Matrice de quantification JPEG.
  int dataQ[] = { 16,11,10,16,24,40,51,61,
                  12,12,14,19,26,58,60,55,
                  14,13,16,24,40,57,69,56,
                  14,17,22,29,51,87,80,62,
                  18,22,37,56,68,109,103,77,
                  24,35,55,64,81,104,113,92,
                  49,64,78,87,103,121,120,101,
                  72,92,95,98,112,100,103,99 };
  CImg<>
    Q = CImg<>(dataQ,N,N)*quality,
    comp(image.width(),image.height(),1,1,0),
    block(N,N), dct(N,N);

  for (int k = 0; k<image.width()/N; ++k)
    for (int l = 0; l<image.height()/N; ++l)
    {
      block = image.get_crop(k*N,l*N,(k + 1)*N-1,(l + 1)*N - 1);
      block -= 128;

      dct = JPEG_DCT(block,cosvalues);
      cimg_forXY(dct,i,j)
        comp(k*N + i,l*N + j) = cimg::round(dct(i,j)/Q(i,j));
    }
  return comp;
}
```

Code source 2: Quantification des coefficients de la *DCT*.

Exercice 3 :

Compression par transformée : Modifiez votre fonction de l'exercice 1 afin d'intégrer le précalcul des valeurs de cosinus du code 1, de manière à la rendre compatible avec le code 2. Appliquez ensuite le traitement défini dans ce code et interprétez le résultat d'un point de vue visuel afin de mieux comprendre les valeurs obtenues en sortie.
Testez votre programme sur l'image `kingfisher.bmp`. Vous devez obtenir, pour tous les blocs de 8×8, des images similaires à la Fig. 3.

Le code 3 illustre alors l'étape de reconstruction des coefficients. Bien entendu, il y a perte d'information entre les étapes de quantification et de reconstruction.

```
/*
  JPEGDecoder: Calcul de l'image reconstruite à partir des DCT

  img_dct   : DCT quantifiées <=> données d'entrée
  quality   : Facteur de qualité ( faible = meilleure qaulité )
  cosvalues : Précalcul des valeurs du cosinus pour la DCT
  return    : Image reconstruite
*/
CImg<> JPEGDecoder(CImg<>& img_dct, float quality, CImg<>& cosvalues)
{
  unsigned int N = 8; // Résolution d'un bloc
  // Matrice de quantification JPEG.
  int dataQ[] = { 16,11,10,16,24,40,51,61,
                  12,12,14,19,26,58,60,55,
                  14,13,16,24,40,57,69,56,
                  14,17,22,29,51,87,80,62,
                  18,22,37,56,68,109,103,77,
                  24,35,55,64,81,104,113,92,
                  49,64,78,87,103,121,120,101,
                  72,92,95,98,112,100,103,99 };
  CImg<>
    Q = CImg<>(dataQ,N,N)*quality,
    decomp(img_dct.width(),img_dct.height(),1,1,0),
    dct(N,N), blk(N,N);

  for (int k = 0; k<img_dct.width()/N; ++k)
    for (int l = 0; l<img_dct.height()/N; ++l)
    {
      dct = img_dct.get_crop(k*N,l*N,(k + 1)*N - 1,(l + 1)*N - 1);
      dct.mul(Q);
      blk = JPEG_IDCT(dct,cosvalues) + 128;
      cimg_forXY(blk,i,j)
        decomp(k*N + i,l*N + j) = blk(i,j);
    }
  return decomp;
}
```

Code source 3: Calcul des coefficients de la *DCT* lors de la reconstruction.

Exercice 4 :

Décompression par transformée : Modifiez votre fonction de l'exercice 2 afin d'intégrer le précalcul des valeurs de cosinus du code 1, de manière à la rendre compatible avec le code 3. Testez le code 3 en l'appliquant après le traitement du code 2.
Testez votre programme sur l'image `kingfisher.bmp`. Le résultat final doit permettre de retrouver une image proche de l'originale, mais avec des différences dues à la phase de compression/décompression et à la perte de données. Vous devez obtenir des résultats similaires à ceux présentés dans la Fig. 4.

4 Algorithme simplifié du *JPEG*

Les pertes d'information dans l'image compressée peuvent s'évaluer par la distorsion $\epsilon = \frac{1}{N \times M} \sum_{i=0}^{N-1} \sum_{j=0}^{M-1} \left(\widetilde{I}[i,j] - I[i,j] \right)^2$, qui correspond à l'écart quadratique moyen entre l'image reconstruite et l'image initiale.

Exercice 5 :

Calcul de la distorsion par écart quadratique moyen : Concevez une fonction permettant de calculer la distorsion entre l'image originale et l'image compressée, dont le prototype est : `float distorsionRate(CImg<>& image, CImg<>& comp_image)`. Testez cette fonction sur deux images issues de votre exercice précédent.

Afin de mettre en évidence la compression par transformée, nous implémentons une version simplifiée de l'algorithme de compression. Nous ne mettrons en place que la transformation des blocs de 8×8 pixels par la *DCT*, la quantification ainsi que la reconstruction de l'image. Ces différentes étapes sont représentées sur la Fig. 3. L'algorithme correspondant est donné en code 4.

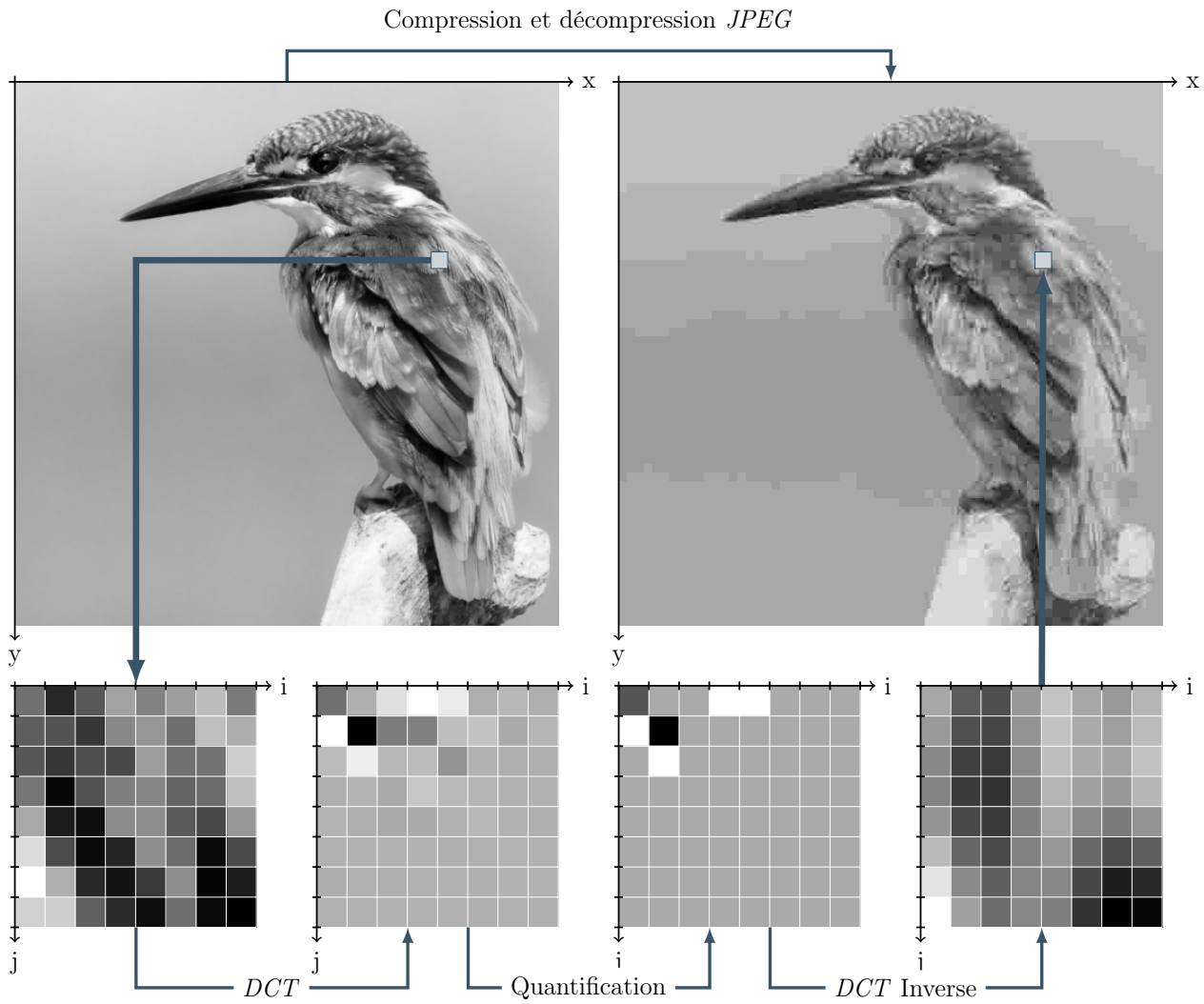

FIGURE 3 – Traitement d'un bloc par la *DCT* dans la compression *JPEG*.

```
int main(int argc,char **argv)
{
  // Passage des paramètres par ligne de commande
  cimg_usage("Compression simplifiée JPEG ");
  const char
    *input_f = cimg_option("-i","lena.bmp","Image d'entrée"),
    *output_f = cimg_option("-o",(char*)0,"Image de sortie");
  float quality = cimg_option("-q",1.0,"Facteur de qualité");

  // Lecture de l'image d'entrée.
  CImg<> imgIn(input_f);
  imgIn.channel(0);

  // Pré-calcul des valeurs des cosinus.
  CImg<> cos_values = genCosValues();

  // Calcul de l'image par DCT + quantification.
  CImg<> dct_image = JPEGEncoder(imgIn,quality,cos_values);

  // Reconstruction de l'image.
  CImg<> comp_image = JPEGDecoder(dct_image,quality,cos_values);

  // Affichage de l'écart quadratique entre les images.
  float dist = distorsionRate(imgIn,comp_image);
  std::cout << "Distorsion Rate : " << dist << std::endl;

  // Affichage des images.
  (imgIn,dct_image,comp_image).display("Image d'entrée - "
                                       "Image des blocs DCT - "
                                       "Image decompressée");

  // Sauvegarde de l'image (si le nom existe).
  if (output_f)
    comp_image.save(output_f);
}
```

Code source 4: Compression par transformée - Version simplifiée du *JPEG*.

La Fig. 4 illustre la compression par transformée en utilisant la *DCT*. L'algorithme qui a été mis en place est une version simplifiée de la compression *JPEG* où le codage entropique, c'est-à-dire le codage de Huffman, n'est pas intégré. On peut sur-estimer le taux de compression par le rapport du nombre de coefficients non nuls dans l'espace *DCT* sur le nombre de coefficients. On peut remarquer que la diminution de la qualité de l'image n'est pas proportionnelle au gain sur la taille de l'information compressée.

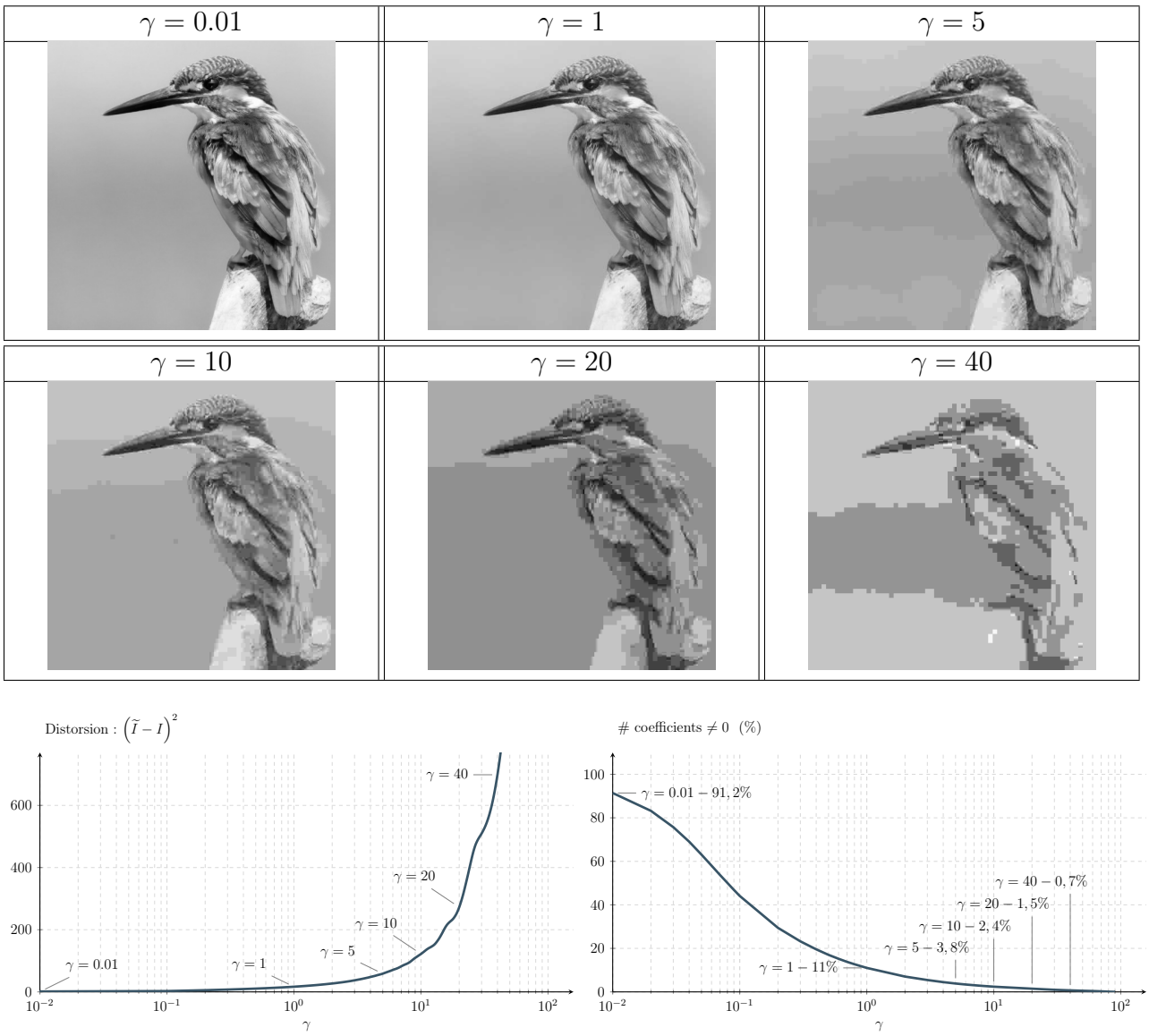

FIGURE 4 – Résultats de la compression *JPEG* en modifiant le paramètre γ. Images reconstruites pour différentes valeurs de γ. Évolution en fonction de γ de la distorsion et du taux de coefficients non nuls dans l'espace *DCT*.

Exercice 6 :

Calcul de la distorsion de la compression JPEG : Concevez une fonction permettant de calculer la distorsion entre l'image originale et l'image compressée pour un ensemble de facteurs de qualité différents, puis affichez la courbe résultante.
Testez votre programme sur l'image `kingfisher.bmp`. Vous devez obtenir des résultats proches de ceux présentés dans la Fig. 4.